09 — Composite Indexes

> *"Column order in a composite index is not a detail — it determines whether the index is used at all."*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Explain why column order is critical in composite indexes
- Apply the leading column rule to determine which queries use a composite index
- Order columns correctly for equality + range queries
- Design composite indexes that also satisfy ORDER BY clauses
- Decide between a composite index and multiple single-column indexes
- Interpret EXPLAIN output to verify composite index usage

What is a Composite Index

A **composite index** (also called a multi-column or compound index) is a B-tree index built on two or more columns.

```
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);
```

PostgreSQL stores index entries sorted first by `customer_id`, and within each `customer_id` value, sorted by `created_at`. This enables efficient access patterns that involve both columns.

ASCII Diagram: Composite Index Sort Order

```
Composite index on orders(customer_id, created_at):

Leaf page entries (sorted lexicographically):
┌──────────────────────────────────────────────────────────────────┐
│ (1, 2024-01-05) │ (1, 2024-03-12) │ (1, 2024-06-01) │            │
│ (2, 2024-01-10) │ (2, 2024-02-28) │                 │            │
│ (3, 2024-01-01) │ (3, 2024-04-15) │ (3, 2024-05-20) │            │
│ (5, 2024-02-14) │                 │                 │            │
│ (7, 2024-03-01) │ (7, 2024-03-15) │ (7, 2024-06-30) │            │
└──────────────────────────────────────────────────────────────────┘
  ↑ sorted by customer_id first, then by created_at within each customer_id

Query 1: WHERE customer_id = 3
  → Navigate tree to first (3, *)
  → Scan forward: (3, 2024-01-01), (3, 2024-04-15), (3, 2024-05-20)
  → Stop when customer_id changes to 4
  ✓ EFFICIENT — contiguous range in index

Query 2: WHERE customer_id = 3 AND created_at > '2024-04-01'
  → Navigate tree to (3, 2024-04-01)
  → Scan forward: (3, 2024-04-15), (3, 2024-05-20)
  → Stop when customer_id changes
  ✓ VERY EFFICIENT — narrow range

Query 3: WHERE created_at > '2024-04-01'
  → No efficient starting point — customer_ids are interleaved
  → Would need to scan ALL entries and filter by date
  ✗ INEFFICIENT — effectively a full index scan
  (Planner would choose Seq Scan instead)

Query 4: WHERE created_at = '2024-01-05' AND customer_id = 1
  → SQL order doesn't matter! Planner reorders predicates
  → Same as Query 2 (equality + equality)
  ✓ EFFICIENT
```

The Leading Column Rule

The **leading column rule** (prefix rule): a composite index on `(a, b, c)` can be used for queries filtering on:

Query Predicates	Uses Index?	Notes
`a = 1`	Yes	Exact prefix
`a = 1 AND b = 2`	Yes	Exact prefix
`a = 1 AND b = 2 AND c = 3`	Yes	Full key
`a = 1 AND c = 3`	Partial	Only `a` used for index scan; `c` filtered

`b = 2`	No	Missing leading column `a`
`b = 2 AND c = 3`	No	Missing leading column `a`
`c = 3`	No	Missing leading columns
`a > 1`	Yes	Range on leading column
`a > 1 AND b = 2`	Limited	Range on `a` stops `b` from being used for scan

Prefix Matching in Detail

```
CREATE INDEX idx_abc ON orders(a, b, c);

-- Can use index for: a, (a, b), (a, b, c)
-- Cannot use index for: b, c, (b, c)

-- IMPORTANT: SQL predicate order doesn't matter, logical prefix does
-- WHERE b = 2 AND a = 1     → uses (a, b) prefix (planner reorders)
-- WHERE c = 3 AND b = 2 AND a = 1 → uses (a, b, c) full key (planner reorders)
```

Column Order Strategy

There are several principles for ordering columns in a composite index. The optimal order depends on your query patterns.

Principle 1: Equality Before Range

When a composite index has both equality and range conditions, **equality columns first**.

```
-- Queries: WHERE customer_id = 42 AND created_at > '2024-01-01'
--          WHERE customer_id = 42 AND status = 'active' AND created_at > '2024-01-
01'

-- GOOD: equality columns first
CREATE INDEX idx_orders_cust_date ON orders(customer_id, created_at);

-- Reason:
-- customer_id=42 narrows to a contiguous range of index entries (all for cust 42)
-- created_at > '2024-01-01' further narrows within those entries
-- Both columns used for the range scan

-- BAD: range column first
CREATE INDEX idx_orders_date_cust ON orders(created_at, customer_id);
-- For WHERE customer_id = 42 AND created_at > '2024-01-01':
-- The range scan on created_at would span ALL customers
-- customer_id = 42 filter applied AFTER, not used to narrow the index scan
```

Principle 2: Most Selective Equality Column First

When you have multiple equality conditions, put the **most selective** (highest cardinality) column first. This minimizes the number of index entries examined before the second column narrows the results.

```
-- Columns: customer_id (200,000 unique values) and status (3 unique values)
-- Query: WHERE customer_id = 42 AND status = 'active'

-- BETTER: high cardinality column first
CREATE INDEX ON orders(customer_id, status);
-- customer_id=42 → ~50 entries → filter for status='active' → ~45 entries

-- WORSE: low cardinality column first
CREATE INDEX ON orders(status, customer_id);
-- status='active' → 3M entries → filter for customer_id=42 → ~50 entries
-- More entries scanned to find same result
```

However, this rule is secondary to Principle 1. Equality always comes before range regardless of selectivity.

Principle 3: Match the ORDER BY

If the query has `ORDER BY`, add those columns (in order) after the equality/range columns. This avoids a separate sort step.

```
-- Query: WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10
CREATE INDEX ON orders(customer_id, created_at DESC);
-- Index provides pre-sorted results → no sort operator needed
-- Planner stops after LIMIT rows found → ultra-fast for small LIMIT

-- Without DESC:
CREATE INDEX ON orders(customer_id, created_at);
-- Planner must sort the results of the index scan → slower for large result sets
```

Equality Before Range

This is the most impactful rule for composite index design.

```
Query: WHERE status = 'active' AND created_at BETWEEN '2024-01-01' AND '2024-06-30'

Scenario A: Index on (status, created_at)
  1. Navigate to first (status='active', created_at >= '2024-01-01')
  2. Scan forward: all (status='active', created_at ...) entries in range
  3. Stop when date exceeds '2024-06-30' OR status changes
  → Both columns used efficiently ✓

Scenario B: Index on (created_at, status)
  1. Navigate to first (created_at >= '2024-01-01')
  2. Scan forward: ALL entries with created_at in range (ALL statuses)
  3. Filter status='active' from each entry
  → Only created_at used for range scan; status only filters
  → More entries scanned (all statuses, not just 'active') ✗
```

The "Range Column Stops the Scan" Rule

Once the index scan reaches a range predicate column, subsequent columns in the index **cannot further narrow the index scan** — they can only be used as post-scan filters.

```
CREATE INDEX ON orders(customer_id, created_at, status);
-- Query: WHERE customer_id = 42 AND created_at > '2024-01-01' AND status = 'active'
-- Index usage:
--   customer_id = 42     → narrows to customer 42's rows (index scan)
--   created_at > date    → further narrows by date (index scan continues)
--   status = 'active'    → CANNOT narrow index scan (after range column)
--                          → applied as Filter after index scan
```

Composite Index for ORDER BY / GROUP BY

Composite indexes can eliminate sort operations for `ORDER BY` and improve `GROUP BY`.

```
-- Index to support: WHERE customer_id = 42 ORDER BY created_at DESC, total DESC
CREATE INDEX ON orders(customer_id, created_at DESC, total DESC);

-- Verify no sort operator:
EXPLAIN SELECT * FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC, total DESC
LIMIT 10;
-- Expected: Index Scan (no Sort node) → stops after 10 rows found
```

GROUP BY with Index

```
-- Index to support GROUP BY without sort:
CREATE INDEX ON orders(customer_id, status);
-- Query: SELECT customer_id, status, COUNT(*) FROM orders GROUP BY customer_id,
status
-- If the leading GROUP BY columns match the index, PostgreSQL can use a Group
Aggregate
-- without sorting (reading index in sorted order)
```

Index Skip Scan (PostgreSQL 17+)

PostgreSQL 17 introduced **index skip scan**, which allows using a composite index even when the leading column is not in the query predicate.

```
-- Index on (customer_id, created_at)
-- Before PG17: query WITHOUT customer_id cannot use this index
-- After PG17: planner can do a "skip scan" that iterates over distinct customer_id
values
```

```
-- PG17+: this may use the composite index via skip scan:
SELECT * FROM orders WHERE created_at > '2024-06-01';
-- Planner enumerates distinct customer_id values, then scans each range
-- Only beneficial when the leading column has few distinct values
```

When Multiple Single-Column Indexes Beat One Composite

PostgreSQL can combine multiple single-column indexes using a **Bitmap And** or **Bitmap Or** operation.

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_date ON orders(created_at);

-- For query: WHERE customer_id = 42 AND status = 'active'
-- PostgreSQL might do:
--   Bitmap Index Scan on idx_orders_customer → set of ctids
--   Bitmap Index Scan on idx_orders_status → set of ctids
--   BitmapAnd → intersection
--   Bitmap Heap Scan

-- This can be competitive with a composite index when:
-- 1. Each single-column index is very selective by itself
-- 2. The query patterns vary (sometimes filter on customer only, sometimes status
only)
-- 3. The composite index would need many combinations
```

Composite vs Multiple Single-Column: Decision Guide

Scenario	Prefer
Query always uses both columns together	Composite (more efficient)
Query sometimes uses only col1, sometimes only col2	Two single-column indexes
Query uses col1 + col2 for sorting (ORDER BY)	Composite
Write-heavy table, minimize index count	Fewer composites
Each column highly selective alone	Single-column may work with BitmapAnd

---

Creating Composite Indexes

```
-- Basic composite
CREATE INDEX CONCURRENTLY idx_orders_cust_date
ON orders(customer_id, created_at);

-- With sort directions
CREATE INDEX CONCURRENTLY idx_orders_cust_date_desc
ON orders(customer_id ASC, created_at DESC);
```

```sql
-- Covering composite (INCLUDE extra columns)
CREATE INDEX CONCURRENTLY idx_orders_cust_status_cov
ON orders(customer_id, status)
INCLUDE (total_amount, item_count);

-- Partial composite
CREATE INDEX CONCURRENTLY idx_orders_active_cust_date
ON orders(customer_id, created_at)
WHERE status IN ('pending', 'processing');

-- Expression in composite
CREATE INDEX CONCURRENTLY idx_users_domain_lower_email
ON users(domain, lower(email));
```

EXPLAIN Output

Good: Composite index used for both columns

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE customer_id = 42 AND created_at > '2024-01-01';
```

```
Index Scan using idx_orders_cust_date on orders
    (cost=0.56..45.89 rows=23 width=128)
    (actual time=0.045..0.234 rows=21 loops=1)
  Index Cond: ((customer_id = 42) AND (created_at > '2024-01-01'))
  Buffers: shared hit=7
Planning Time: 0.189 ms
Execution Time: 0.267 ms
```

Both columns appear in `Index Cond` — both used for the index scan.

Partial: Only first column used for scan

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE customer_id = 42 AND status = 'active';
-- Index on (customer_id, created_at) — status not in index
```

```
Index Scan using idx_orders_cust_date on orders
    (cost=0.56..89.34 rows=15 width=128)
    (actual time=0.034..0.789 rows=14 loops=1)
  Index Cond: (customer_id = 42)       ← only customer_id used for scan
  Filter: (status = 'active')           ← status applied as post-scan filter
  Rows Removed by Filter: 7
```

This is a signal that the index may not be optimal — consider adding `status` to the composite index or creating a separate index.

Common Mistakes

1. Putting the range column before equality columns

```
-- WRONG for queries: WHERE status = 'active' AND created_at > '2024-01-01'
CREATE INDEX ON orders(created_at, status);
-- Range on created_at first → status can only be a post-scan filter

-- CORRECT:
CREATE INDEX ON orders(status, created_at);
```

2. Ignoring column order and hoping the planner sorts it out

The planner does NOT reorder columns within the index structure. It can reorder predicates in the SQL query, but the physical index sort order is fixed at creation time.

3. Creating (a, b) and (a) separately

```
-- REDUNDANT: (a) is already covered by (a, b)
CREATE INDEX ON orders(customer_id, created_at);
CREATE INDEX ON orders(customer_id);  -- unnecessary!
-- The composite index (customer_id, created_at) can be used for
-- queries filtering only on customer_id (leading column prefix)
```

4. Too many columns in composite index

A 5-column composite index rarely helps queries that don't filter on the first 2-3 columns. It also adds significant write overhead and storage.

5. Not testing the actual query

Always use `EXPLAIN (ANALYZE, BUFFERS)` to verify your index is used and how.

Best Practices

1. **Equality columns first, range columns after** — the most important rule.
2. **Most selective equality column first** among equality columns.
3. **Match ORDER BY** by including sort columns (with direction) at the end.
4. **Drop the single-column index** on column `a` when you have a composite `(a, b)` — the composite handles `a`-only queries too.
5. **Test with realistic data and query volumes** — the planner's choice depends on table statistics.
6. **Use EXPLAIN ANALYZE** to verify index usage and that both `Index Cond` entries are present (not moved to `Filter`).

7. **Limit composite width** — rarely more than 3-4 columns provides benefit. Wider indexes have diminishing returns and higher write costs.

Performance Considerations

Index Entry Size

Each additional column in a composite index increases the size of each leaf entry.

```
Single-column on customer_id (4 bytes): ~10 bytes per entry
Composite (customer_id + created_at): ~18 bytes per entry
Composite (customer_id + created_at + status): ~22 bytes per entry
```

Larger entries mean fewer entries per page → more pages to read → higher I/O.

Write Amplification

Each composite index is updated once per write (same as single-column). So `(a, b, c)` does not cost 3× the writes of `(a)` — only once, for the composite.

However, it does occupy more space per entry, meaning more page I/O for rebuilds.

Interview Questions & Answers

Q1. What is the leading column rule for composite indexes?

A: The leading column rule states that a composite index on columns (a, b, c) can only be used by queries that include a predicate on the leading column or columns as a prefix. Queries filtering on `a` alone, `a` and `b`, or `a`, `b`, and `c` can use the index. Queries filtering on `b` alone, `c` alone, or `b` and `c` (without `a`) cannot use the index for an efficient scan because the index entries are sorted by `a` first — without knowing `a`, there's no efficient starting point.

Q2. Why should equality conditions come before range conditions in a composite index?

A: When the index scan reaches a range condition on column N, the scan produces a contiguous range of entries in the index. But within that range, all possible values of column N+1 are interleaved (for different values of column N). So the range condition "uses up" the index's ability to filter by subsequent columns — they can only be applied as post-scan filters. By placing equality conditions first, each equality condition narrows to a smaller, contiguous subset of entries, and the following range condition can then precisely delimit that subset. The equality columns narrow the range; the range column further refines it.

Q3. If you have a composite index on (a, b), do you also need a separate index on (a)?

A: No. The composite index on (a, b) can be used for queries filtering only on `a` because `a` is the leading column. The planner will use the composite index for a range/equality scan on just `a`, reading the entries in order. A separate index on just `a` would be redundant and should be dropped to reduce write overhead and storage.

Q4. How do you design a composite index to avoid a sort operation?

A: Add the ORDER BY columns (in the same order and direction) after the WHERE condition columns. For example, for query `WHERE customer_id = 42 ORDER BY created_at DESC`: `CREATE INDEX ON orders(customer_id, created_at DESC)`. The equality condition on `customer_id` narrows to a contiguous range, and within that range entries are already sorted by `created_at DESC` — matching the ORDER BY exactly. The planner can return results without a separate Sort node, which is crucial when combined with LIMIT.

Q5. When might two single-column indexes be better than one composite index?

A: When queries have varying filter patterns: sometimes filtering on column A alone, sometimes on column B alone, and sometimes on both together. A composite index (A, B) handles A-only and (A, B) queries but NOT B-only queries. Two single-column indexes handle all three patterns. The planner can also combine two single-column indexes via BitmapAnd for the (A, B) case, though usually a composite is more efficient for frequent A+B queries. The tradeoff: two indexes means more storage and write overhead.

Exercises with Solutions

Exercise 1

Given query: `SELECT * FROM orders WHERE region = 'US' AND amount > 500 ORDER BY created_at DESC LIMIT 20` Design the optimal composite index.

Solution:

```
-- Analysis:
-- Equality: region = 'US' → first column
-- Range: amount > 500 → second column (after equality)
-- ORDER BY: created_at DESC → third column (after range, for sort elimination)

CREATE INDEX CONCURRENTLY idx_orders_region_amount_date
ON orders(region, amount, created_at DESC)
WHERE amount > 0;  -- optional partial if amount is always positive

-- Why this works:
-- 1. region='US' narrows to US orders
-- 2. amount > 500 narrows to large orders
-- 3. created_at DESC provides pre-sorted results → no Sort node
-- 4. LIMIT 20 → stops after 20 results → very fast

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE region = 'US' AND amount > 500
ORDER BY created_at DESC LIMIT 20;
```

Production Scenarios

Scenario 1: Multi-Tenant SaaS Application

Each API query includes `tenant_id`. Additional filters vary by endpoint.

```
-- Common query patterns:
-- 1. GET /api/orders?tenant=T1&status=pending
-- 2. GET /api/orders?tenant=T1&date_from=2024-01&date_to=2024-03
-- 3. GET /api/orders?tenant=T1 (sorted by created_at DESC, paginated)

-- Composite index strategy:
-- Index 1: Covers patterns 1 and 3
CREATE INDEX CONCURRENTLY idx_orders_tenant_status_date
ON orders(tenant_id, status, created_at DESC);

-- Index 2: Covers pattern 2 (range on date)
CREATE INDEX CONCURRENTLY idx_orders_tenant_date
ON orders(tenant_id, created_at DESC);
-- (Also covers pattern 3)

-- Result: tenant_id is always the leading column (required by all queries)
-- Pattern 1: uses idx_orders_tenant_status_date (equality + equality + sort)
-- Pattern 2: uses idx_orders_tenant_date (equality + range)
-- Pattern 3: uses idx_orders_tenant_date (equality + sort)
```

Cross-References

- 02_btree_indexes.md — B-tree structure underlying composites
- 08_partial_indexes.md — Combining partial with composite
- 10_covering_indexes.md — INCLUDE clause for composite
- ../08_Query_Optimization/04_join_algorithms.md — Join algorithms and index usage